

<i>Element</i>	<i>Decyzja</i>	<i>Kontekst</i>	<i>Alternatywy</i>	<i>Uzasadnienie</i>	<i>Trade-offy</i>
Baza danych	Relacyjna baza danych PostgreSQL	Sklep przechowuje informacje o grach, użytkownikach, zamówieniach. Istnieją między nimi relacje, musimy zachować jednolitość danych.	NoSQL, np. MongoDB – dane są relacyjne, lepiej wybrać bazę ze zdefiniowanym z góry schematem dla przejrzystości kodu SQLite – szybki i łatwy, ale końcowy projekt zakłada konteneryzację osobnych serwisów	Ten wybór obsługuje klucze obce przydatne w projekcie, ACID, zabezpiecza płatności	Trzeba uruchomić kontener (więcej zasobów niż SQLite dla developmentu), trochę wolniejsza niż MySQL
ORM i migracje	Prisma	Potrzebujemy ORMa do type safe migracji. Backend jest w TypeScript, Prisma jest dobrze zintegrowana	Drizzle ORM – nastawiony na bardziej tradycyjne zapytania w SQL Surowy SQL (+ node-postgres) – większa kontrola i prędkość, ale więcej bugów i brak generacji typów TypeORM – gorszy DX, nie jest już obsługiwany	Dobre type safety, dobry DX, dobra produktywność	Surowy SQL do skomplikowanych zapytań (pewnie nie będzie potrzebny)
Backend API	tRPC	Projekt full-stack potrzebuje komunikacji serwera z klientem – gry, szczegóły, koszyk, płatności	REST – standard, brak dzielenia typów między serwerem a klientem (duplikacja) GraphQL – w-sam-raz fetching, ale trochę trudniejszy, drużyna nie jest zaznajomiona, jego elastyczność i możliwości	Prostota i świetny DX, bezpieczeństwo typów, monorepo w Typescript idealnie współgra z tRPC	Brak natywnego cache'owania (zajmuje się tym TanStack po stronie klienta), ciężko testować prostym curl'em

			nie są wymagane dla małego projektu		
Autentykacja	JWT, access token jako httpOnly i refresh token w bazie danych	Projekt musi rozróżniać zalogowanych użytkowników i chronić niektóre endpointy	Sesje – serwer korzysta z pamięci albo Redis, łatwiejsza inwalidacja, ale wymaga dzielonego przechowywania sesji OAuth – bezpieczne, ale wprowadza zależność od zewnętrznych usług	Dwa tokeny rozwiązują problem inwalidacji. 15-minutowy access token przechowywany jako stan React chroni przed XSS	Access token jest tracony przy odświeżaniu, wymagany fetch po refresh token
Frontend	Next.js (App Router) i komponenty serwera, @trpc/react-query i komponenty klienta	Sklep z gramami posiada dwa rodzaje widoków: publiczne z dużą ilością odczytów jako komponenty serwera (lista gier) i chronione, interaktywne komponenty klienta (zakupy, koszyk)	React SPA – łatwiejszy, ale mniej wydajny dla content-heavy stron sklepu Remix – mniejszy ekosystem, drużyna nie jest zaznajomiona	Katalog gier i szczegóły są wspólne dla wszystkich, SSR skraca postrzegany czas ładowania. Next.js przechwytuje token i może przekierować przed załadowaniem chronionej strony	Trzeba pamiętać o „use client” dla komponentów, trzeba ustawić dwie konfiguracje trPC
Konteneryzacja	Dwa serwisy w Dockerze: app (Next.js, frontend i backend) oraz db (postgresql)	Wymaganie projektowe opisują konteneryzację, Next.js umożliwia jeden serwis backend/frontend	3 serwisy – podejście SPA, nie mamy oddzielnych środowisk dla front/end Kubernetes – przekomplikowane Serverless (np. Supabase) – nie można uruchomić lokalnie jedną komendą	Nie potrzebujemy Nginx, CORS, osobnego builda dla API.	Wymaga przebudowy aplikacji przy zmianie jednego z endów